Mortgage / Loan SaaS on .NET + Azure

Architecture Notes from Our Discussion

---

1. Core scalability challenges in mortgage / loan-based SaaS

Mortgage and loan SaaS platforms do not fail at scale because of simple web traffic alone. The deepest problems are usually around business-process complexity, regulatory traceability, and document-heavy workflows.

1.1 Long-running, stateful workflows

A loan is not a short request/response transaction. It can live for weeks or months and pass through intake, disclosures, credit, underwriting, appraisal, conditions, closing, funding, and servicing handoff.

Why this is hard

- Many stages, retries, exceptions, and manual interventions
- Workflows span days or weeks
- Side effects matter, so replay/retry must be controlled

How it is addressed

- Durable workflow orchestration
- Explicit state machines
- Event-driven processing with idempotency
- Clear separation between workflow, decisioning, and side effects

---

1.2 Document-heavy ingestion and extraction

Mortgage systems process pay stubs, W-2s, bank statements, tax returns, disclosures, title docs, and closing packages.

Why this is hard

- OCR and extraction are compute-heavy
- Docs are inconsistent, duplicated, partial, or poor quality

- Downstream underwriting depends on extracted accuracy

How it is addressed

- Async pipelines
- Queues and backpressure
- OCR/classification/extraction split into stages
- Confidence scoring with human-in-the-loop fallback
- Provenance tracking from extracted field back to source doc

---

1.3 Rules complexity and drift

Loan eligibility, pricing, disclosures, investor overlays, state rules, and compliance rules create combinatorial logic sprawl.

Why this is hard

- Rules get encoded in code, configs, spreadsheets, and tribal knowledge
- New products and jurisdictions increase complexity
- Logic becomes opaque and difficult to test

How it is addressed

- Centralized rules/decision engines
- Versioned policy logic
- Explainability
- Large scenario-based regression suites

1.4 External dependency bottlenecks

Mortgage SaaS depends on third parties such as credit bureaus, appraisal, title, fraud checks, income verification, e-sign, and AUS systems.

Why this is hard

- Third-party APIs are often slow, rate-limited, or unstable
- Your throughput can be capped by partner availability

- Legacy provider interfaces may be awkward or semi-manual

How it is addressed

- Provider abstraction layers
- Circuit breakers and retries
- Async integration patterns
- Bulkhead isolation
- Operational degraded-mode playbooks

1.5 Data consistency vs throughput

Loans are touched by borrowers, loan officers, processors, underwriters, and automated systems simultaneously.

Why this is hard

- Race conditions create compliance and financial risk
- Some data can be eventually consistent, some cannot
- Many actors update the same logical loan file

How it is addressed

- Strong domain boundaries
- Versioned aggregates
- Optimistic concurrency
- Sagas/compensating transactions
- Immutable audit events

1.6 Regulatory and audit scalability

At scale, every material decision may need to be reconstructed months later.

Why this is hard

- Need to know who changed what and when

- Must know what rule version and source data were used
- Need audit-ready evidence packages

How it is addressed

- Full lineage for decisions, data, and documents
- Time-versioned records
- Immutable evidence stores
- Separation of operational and audit data planes

1.7 Multi-tenancy with deep customization

Different lenders, brokers, or partners often want different workflows, branding, and approval paths.

Why this is hard

- Customization pressure creates a forked platform
- Tenant-specific logic can infect the core system
- Enterprise customization debt accumulates

How it is addressed

- Strong extension/configuration model
- Tenant-specific metadata, not code forks
- Isolated config and entitlements
- Governed feature flags and workflow templates

1.8 Peak-load asymmetry

Load spikes are tied to month-end, quarter-end, lock expirations, rate changes, and macro events.

Why this is hard

- User and batch workloads collide
- Peak traffic aligns with business-critical deadlines

- Average utilization is misleading

How it is addressed

- Queue-based load leveling
- Separate online and batch compute
- Priority scheduling
- Precomputation and caching
- Business-event-based capacity planning

1.9 Search, reporting, and analytics

Lenders need operational dashboards, compliance reporting, queue views, fallout analysis, and productivity metrics.

Why this is hard

- OLTP systems are poor analytics systems
- Search-heavy UIs can become expensive
- Reporting competes with production workloads

How it is addressed

- Separate analytical plane
- CDC/event streaming into reporting stores
- Materialized views
- Search indexes
- Bounded freshness rather than fake “real time”

1.10 Security and PII isolation

Loan systems handle SSNs, income docs, bank statements, credit data, and signatures.

Why this is hard

- More integrations and tenants increase attack surface

- Sensitive data can leak into logs, caches, or queues
- AI/document systems can widen exposure

How it is addressed

- Data minimization
- Tokenization/vaulting for sensitive fields
- Field-level access control
- Segmented storage and redacted telemetry
- Strong identity and network isolation

1.11 Servicing/payment correctness

If servicing is included, payment posting, escrow, delinquency, and reconciliation become critical.

Why this is hard

- Money movement requires correctness across retries and reversals
- Out-of-order events can corrupt state
- Ledger errors are catastrophic

How it is addressed

- Ledger-style models
- Immutable journals
- Correction entries rather than destructive updates
- Reconciliation pipelines
- Idempotent payment processing

1.12 AI and automation with exception handling

AI helps with docs, support, fraud, and recommendations, but exception handling becomes the real bottleneck.

Why this is hard

- Automation confidence varies
- Edge cases pile up in manual queues
- Humans become the scaling ceiling

How it is addressed

- Confidence thresholds
- Straight-through lanes vs exception lanes
- Feedback loops from manual corrections
- Exception aging and touch-rate measurement

---

2. Reference architecture on .NET + Azure

A good mental model is:

channel layer → API layer → domain services → workflow/events → data/documents → analytics/compliance → security/ops

---

2.1 Front-end / channel layer

These are the user-facing experiences.

Examples

- Borrower portal
- Internal loan officer UI
- Processor/underwriter console
- Broker/partner portal
- Admin/configuration portal

Typical stack

- Front-end SPA or web app
- ASP.NET Core backends
- Azure App Service or AKS for hosting

- Front Door / Application Gateway at the edge

Responsibilities

- UI delivery
- request routing
- auth/session handoff
- internet-facing scale and protection

2.2 Identity and access

Identity is foundational in a lending platform.

Typical Azure components

- Microsoft Entra ID for workforce identities
- External identity model for borrowers
- Managed Identity for service-to-service auth
- Azure RBAC for infrastructure access

Responsibilities

- borrower login
- internal staff access
- role/permission enforcement
- service authentication without embedded secrets

2.3 API layer

This is the controlled entry point to the platform.

Typical stack

- ASP.NET Core Web API
- Azure API Management (APIM)

Responsibilities

- auth enforcement
- rate limiting
- request/response transformation
- versioning
- partner API exposure
- developer portal support

2.4 Core domain services

These should align with bounded contexts.

Examples

- Application Intake Service
- Borrower Profile Service
- Loan Service
- Pricing and Eligibility Service
- Conditions Service
- Disclosure Service
- Closing/Funding Service
- Servicing Handoff Service
- Document Metadata Service
- Audit/Compliance Service

Hosting choices

- Azure App Service for simpler estates
- AKS for larger microservice estates

2.5 Workflow orchestration

Long-running loan workflows should be orchestrated explicitly.

Azure fit

- Azure Durable Functions
- Message-driven background workers

Responsibilities

- manage state over time
- wait for external responses
- schedule retries
- track outstanding work
- handle timeouts and completion rules

2.6 Messaging and events

Messaging should separate commands from notifications.

Typical Azure components

- Azure Service Bus
- Azure Event Grid

Service Bus is best for

- reliable commands
- durable business jobs
- dead-lettering and retries
- point-to-point or topic-based enterprise messaging

Event Grid is best for

- reactive event fan-out
- Azure-native events such as blob uploaded
- integration off storage/object events

2.7 Data stores

Different workloads need different stores.

Azure SQL

- core relational loan data
- transactional integrity
- strongly relational operational state

Cosmos DB

- JSON-heavy metadata
- high-scale document/config/event workloads
- draft application or flexible aggregates

Azure Cache for Redis

- low-latency hot data
- config cache
- session/cache acceleration

Blob Storage

- mortgage documents and artifacts

2.8 Document processing plane

A separate subsystem is needed for documents.

Typical pipeline

1. document uploaded to Blob Storage
2. storage event emitted
3. Function or worker consumes event
4. OCR/classification/extraction runs
5. validation/enrichment occurs
6. extracted values and provenance are stored
7. workflow resumes

Useful components

- Blob Storage
- Event Grid
- Azure Functions / worker services
- Azure AI Document Intelligence
- metadata store
- optional AI Search

2.9 Background processing

Not every operation should be synchronous.

Typical uses

- OCR jobs
- doc conversion
- outbound partner polling
- nightly QC checks
- disclosure package generation
- batch exports

Typical Azure components

- Azure Functions
- containerized .NET worker services
- Durable Functions when stateful coordination is needed

2.10 Analytics and reporting

Analytics should be separated from the OLTP system.

Typical stack

- ingestion/ETL pipeline

- warehouse / analytics store
- Power BI

Use cases

- pipeline metrics
- condition aging
- underwriting productivity
- lock pull-through
- compliance reports
- investor delivery metrics

2.11 Search

Loan operations teams live inside filtered searches and queue views.

Useful capabilities

- loan search by attributes
- doc metadata search
- extracted field search
- note/task/condition search

Good fit

- Azure AI Search

2.12 Security and network isolation

This is critical for lending systems.

Key services

- Azure Key Vault
- Managed Identity
- Private Endpoints / Private Link

- VNet segmentation
- WAF-capable ingress

Responsibilities

- secret management
- encryption key handling
- network isolation for SQL, Storage, and other PaaS services
- reduced public attack surface

2.13 Observability and operations

You need end-to-end telemetry.

Useful components

- Azure Monitor
- Application Insights
- Log Analytics

What to track

- API latency
- queue depth
- external dependency failures
- workflow failures
- retry behavior
- distributed tracing with correlation IDs

2.14 DevOps and release

Typical stack

- Azure DevOps or GitHub Actions
- Azure Container Registry

- IaC via Bicep or Terraform

Responsibilities

- build and package .NET services
- deploy web apps, APIs, functions, or containers
- manage infra reproducibly
- enable safe release patterns

---

3. What BFF means

BFF = Backend for Frontend

A BFF is a backend layer tailored to a specific client experience.

Instead of one giant generic API used by every UI, you create a backend optimized for:

- borrower portal
- internal ops UI
- processor/underwriter console
- partner portal
- mobile client

What a BFF does

- request shaping
- response aggregation
- client-specific auth/session logic
- workflow support for UX
- reducing client chattiness

What it should not do

- become the true domain system of record
- permanently own core underwriting logic
- turn into a second monolith

Loan SaaS examples

Borrower Application BFF

- receives application wizard steps
- saves draft progress
- calls identity/eligibility services
- returns one composite response for the borrower UI

Ops BFF

- aggregates loan summary, borrower data, conditions, disclosures, tasks, and status for the internal user interface

---

4. Initial loan application eligibility / background check flow

This was the core architecture question.

Key principle

Service Bus does not decide whether execution is parallel or sequential.
That decision belongs to the workflow/orchestration design.

4.1 Three possible patterns

Pattern A: Independent commands in parallel

Examples:

- RunCreditCheck
- RunFraudCheck
- RunIdentityVerification
- RunOFACCheck
- RunIncomeVerification
- RunEmploymentVerification

These commands are placed on Service Bus and handled independently by different services.

Good for

- independent checks
- retry isolation
- elastic scaling
- reduced end-to-end latency

Pattern B: Orchestrator fan-out / fan-in

This is usually the best fit for lending workflows.

Example

1. Application submitted
2. Create application/loan record
3. Fan out parallel checks
4. Wait for required results
5. Aggregate outcomes
6. Apply eligibility rules
7. Advance workflow

Why it is strong

- one place owns readiness
- one place owns timeout logic
- one place owns retry/gating logic
- easier to understand than pure choreography

Pattern C: Pure event choreography

Example:

- ApplicationSubmitted event is published
- multiple services subscribe and act

- result events are emitted
- another component figures out when enough is complete

Why this gets messy

- unclear ownership of state
- hard to know when all required work is done
- timeout handling becomes scattered
- duplicate decisions and race conditions become more likely

For loan workflows, this is usually weaker than explicit orchestration.

4.2 How to decide parallel vs sequential

Run in parallel when

- tasks are independent
- one task's output is not needed to start another
- checks hit different downstream providers
- total elapsed time matters
- no business/compliance rule requires strict order

Run sequentially when

- task B depends on task A
- a gating rule must be satisfied first
- expensive downstream checks should be avoided unless earlier gates pass
- deterministic ordering matters
- side effects require strict control

4.3 Typical loan-intake staging model

Parallel group 1

Usually safe early checks:

- identity verification
- fraud check
- OFAC/sanctions/watchlist check
- duplicate application detection
- device/behavioral risk scoring

Parallel group 2

Only if group 1 passes:

- credit pull
- income verification
- employment verification
- pricing/prequalification pull

Sequential group 3

After required results arrive:

- aggregate results
- run eligibility rules
- produce initial decision
- move to next workflow stage

This staged model is usually better than:

- running everything sequentially
- or blasting all checks immediately

4.4 Commands vs events

Command

A command means:
"You, specific service, do this."

Examples:

- RunCreditCheck
- PerformFraudScreen
- GenerateInitialEligibilityDecision

Use commands when:

- exactly one service should act
- the action is required
- ownership is explicit

Event

An event means:
“This happened.”

Examples:

- ApplicationSubmitted
- CreditCheckCompleted
- FraudScreenFailed
- EligibilityDecisionGenerated

Use events when:

- other services may care
- you are broadcasting a fact
- subscribers act independently

Clean design rule

- use **commands** to initiate work
- use **events** to communicate outcomes and state changes

4.5 Recommended pattern for LoanDepot-style intake

- Borrower BFF handles UI flow
- Application Service persists the application

- Workflow orchestrator decides staging and dependencies
- Service Bus dispatches commands to independent check services
- Results are published back
- Orchestrator waits, aggregates, retries, and decides completion
- Final eligibility decision is produced only after required checks converge

---

5. Azure Functions vs Durable Functions duration limits

5.1 Regular Azure Functions

Maximum duration depends on hosting plan.

Consumption plan

- default timeout: 5 minutes
- maximum timeout: 10 minutes

Flex Consumption / Premium / Dedicated

- no hard function timeout enforced by the platform in the same way
- but platform events such as scale-in and host shutdown still matter

HTTP-triggered functions

- practical response window is around 230 seconds due to Azure Load Balancer idle timeout
- so long-running HTTP work should not be done inline

5.2 Durable Functions

Durable Functions are different.

Overall orchestration

- can run for very long periods
- potentially days or weeks
- because state is checkpointed and resumed

Important caveat

- individual orchestrator executions and activity functions still obey the timeout behavior of the hosting environment
- the long-running capability applies to the orchestration as a whole, not a single uninterrupted in-memory execution

Design implication

For loan workflows:

- use Durable Functions for the long-running multi-step process
- keep each activity bounded and retryable
- do not wait synchronously via HTTP for end-to-end completion

---

6. What an index is

An **index** is a data structure that makes data retrieval faster.

A good analogy is the index in the back of a book:

- without it, you scan every page
- with it, you jump directly to relevant content

In databases

An index is created on one or more columns so queries can find rows faster.

Benefits

- faster filtering
- faster searching
- faster sorting
- faster joins

Costs

- uses extra storage
- adds overhead to inserts/updates/deletes
- too many indexes can harm write performance

Common types

- clustered index
- nonclustered index
- composite index
- unique index

In loan SaaS

Useful indexes often exist on:

- LoanNumber
- BorrowerId
- ApplicationId
- TenantId
- Status
- CreatedDate
- CorrelationId

For example, if ops users query:

- loans by tenant
- filtered by status
- sorted by created date

then a composite index like:

(TenantId, Status, CreatedDate)

may be very helpful.

---

7. Practical architecture recommendations

7.1 Good initial Azure stack for a serious lending platform

A pragmatic initial version might use:

- ASP.NET Core

- Azure App Service
- Azure SQL
- Blob Storage
- Azure Service Bus
- Durable Functions
- Event Grid
- Key Vault
- API Management
- Application Insights
- Document Intelligence

This is usually better than starting with a large AKS estate too early.

7.2 When AKS becomes justified

AKS makes more sense when:

- service count is high
- teams need independent release cadences
- platform engineering maturity exists
- workload types are heterogeneous
- Kubernetes-level networking/observability controls are needed

---

7.3 Clean architectural separation

A good mental model is:

- **BFF**: channel-specific API aggregation
- **Domain services**: own business capabilities
- **Orchestrator**: decides sequence, fan-out, fan-in, retries, timeouts
- **Service Bus**: reliable transport for commands/jobs

- **Events**: communicate what happened
- **Blob + doc pipeline**: handle document-heavy workloads
- **Audit plane**: immutable evidence and lineage
- **Analytics plane**: reporting without stressing OLTP systems

---

8. One-line takeaways

Mortgage / loan SaaS scaling

The deepest challenge is not raw compute. It is **business-process entropy under compliance pressure**.

BFF

A **Backend for Frontend** is a client-specific backend layer optimized for a particular UI or channel.

Service Bus

Service Bus is the **transport backbone**, not the business brain that decides dependency order.

Orchestration

Parallel vs sequential execution should be decided by an **orchestrator or workflow design**, not by queue semantics alone.

Durable Functions

Durable Functions are good for **long-running, stateful workflows**, but each execution slice still follows host/runtime constraints.

Index

An index is a **lookup acceleration structure** that speeds retrieval at the cost of storage and write overhead.

---

9. Suggested final reference flow for LoanDepot-style initial intake

1. Borrower submits application via Borrower BFF
2. Application Service stores application and correlation/workflow IDs

3. Orchestrator starts workflow
4. Service Bus dispatches parallel group 1 checks:
    - identity
    - fraud
    - OFAC
    - duplicate detection
5. Orchestrator waits for results
6. If group 1 passes, dispatch group 2:
    - credit
    - income
    - employment
    - pricing/prequal
7. Results return as completion events/messages
8. Eligibility service aggregates and rules engine evaluates
9. Initial eligibility decision is generated
10. Borrower BFF / internal UI receives status and next-step payload

---

10. Final view

For a company like LoanDepot, the strong architecture is not just “microservices on Azure.” It is:

- durable workflows
- clear command/event boundaries
- staged parallelism
- document-native processing
- strong auditability
- careful tenant/config isolation

- explicit BFFs for borrower and ops channels

That is what keeps a loan platform understandable and scalable as volume, regulation, and product complexity grow.

If you want, I can turn this into a **cleaner executive summary page**, a **system design interview answer**, or a **Notion page with collapsible sections and callout-style formatting**.